Configuration Testing

Task Manager

Enrique S. Castro
IT-355 - Software Testing
Alex Mbaziira, PhD
Marymount University - Fall 26'

Table of Contents

Introduction and Testing Environment 3

Testing Environment 3

Tested Configurations 3

Configurations & Prioritized Testing Matrix 4

Risk-Based Selection 5

Testing Scope and Limitations 5

Test Observations 6

Documented Evidence 8

Issue Documentation 11

Summary of Findings 14

Reflection and Conclusion 14

Introduction and Testing Environment

The purpose of this exercise was to develop a simple web application using generative artificial intelligence and evaluate its functionality across different software and hardware configurations. The application developed for the exercise is a trivial task manager. It is written in HTML, CSS, and vanilla JavaScript. It permits users to create, complete, and delete tasks while using the browsers localStorage to preserve task information between sessions.

Configuration testing was performed to evaluate how to app responds to differences in operating systems, device types, screen dimensions, browser settings, and storage availability. Particular attention was given to responsiveness of the design, data persistence, and the app's ability to handle restricted browsers functionality without unexpected failures.

Testing Environment

Testing was performed using Safari on simulated iOS and iPadOS devices. Xcode Device Hub to provide mobile and tablet testing environments. Safari Web Inspector was used to review the applications's HTML, CSS, and JavaScript executions, and locally stored data.

Tested Configurations

Environment	Device	Browser	Configuration
macOS	Mac	Safari	Desktop, JavaScript enabled and
iOS 27.0	iPhone 18 Pro	Safari	Portrait and Landscape, light and dark modes
iPadOS	iPad Pro 13-in (M5)	Safari	Tablet layout and orientation
macOS	Mac	Safari	Simulated blocked local storage

The application was locally hosted using python's built-in http server, allowing the same html file to be accessed across the simulated devices without modifying the application or deploying it to an external server. Two primary testing tools were used:

- **Xcode Simulator:** provided simulated iPhone and iPad environments for evaluating the responsive layouts, changes in orientation, and application functionality.
- **Safari Web Inspector:** Provided access to the document structure, browser console, and local storage. It was also used to simulate storage-write failures and observe application error handling behaviors.

Testing consisted of manually creating, completing, and deleting tasks; manual inspection of stored JSON data; evaluating responsive layouts; and modification of browser configurations. Screenshots were captured to document the application's behavior under the tested conditions.

Configurations & Prioritized Testing Matrix

Configuration Matrix

Configuration Parameter	Supported Values	Test
Browser	Safari, Safari (iOS/iPadOS)	Check task creation, deletion, completion, persistence across desktop and simulated devices
Browser Version	27.0.1 (MacOS); Safari iOS; iPadOS 27.0	Verify application functionality across Safari environments
Screen Size	Desktop(3456 x 2234), iPhone (2622 x 1206), iPad (2064 x 2752)	Evaluate layouts, readability, and control accessibility across device sizes and orientations.
Screen Orientation	Portrait, Landscape	Check responsive layout, readability of interactive controls when device orientation changes
JavaScript Enabled	Yes/No	Verify normal operations when enabled, and response when disabled
Local Storage Support	Available, Blocked, Unavailable	Verify persistence when storage is available, graceful degradation when writes fail
Network Speed	Localhost (network speed not varied)	Document app's local hosting environment; network throttling was not performed
Color Scheme	Light mode, Dark Mode	Evaluate readability, contrast, interface behavior under appearance settings

Prioritized Test Configurations

ID	Browser	Operating System	Device/Configuration	Priority	Status
CT-01	Safari	macOS	Desktop, JavaScript Enabled	High	Tested
CT-02	Safari	iOS 27.0	iPhone 18 Pro, mobile layout	High	Tested
CT-03	Safari	iPadOS	iPad Pro 13-inch (M5), tablet Layout	Medium	Tested
CT-04	Safari	macOS	Desktop, simulated blocked localStorage writes	High	Tested
CT-05	Safari	macOS	Desktop, Javascript disabled	Medium	Tested

Risk-Based Selection

The configurations prioritized conditions that could directly affect the app's core functionality. Desktop Safari establishes a baseline for task management and persistence. The iPhone and iPad environments evaluate the responsive behavior across mobile and tablet devices, this includes orientation and appearance changes.

Storage restrictions were assigned high priority because the application relies on localStorage to preserve user data. Javascript-disabled testing was also included because the application's core functionality depends on JavaScript execution.

The configurations were selected to examine reliability, responsiveness, and graceful degradation without pursuing exhaustive testing of every possible browser and OS combination.

Testing Scope and Limitations

Testing was limited to Safari across macOS, iOS, and iPadOS environments. While the application was evaluated across different devices, screen orientations, and appearance settings, cross-browser compatibility and historical browser versions were not tested. Network performance testing was also excluded because the application was hosted locally and did not depend on external resources or services. These limitations reflect the scope of the selected configurations, with testing focused on responsive design, JavaScript execution, and browser storage availability.

Test Observations

CT-01 – Desktop Functionality and Persistence

The application was tested using Safari on macOS to establish a baseline for functionality. Tasks were created, marked as completed, and deleted. Safari Web Inspector was used to inspect the contents of the browser's localStorage through the JavaScript console and Storage panel.

The stored JSON contained task descriptions, unique identifiers, and completion states corresponding to the information displayed in the application. The application also retained previously stored tasks after JavaScript was re-enabled.

CT-02 – iPhone Responsiveness

Testing was conducted using an iPhone 18 Pro simulator running iOS 27.0. The application was evaluated in portrait and landscape orientations and under light and dark appearance settings.

In portrait orientation, the task input field and Add Task button were arranged vertically. In landscape orientation, the controls appeared horizontally. Task creation, completion, and deletion remained functional, although the reduced vertical space in landscape orientation required scrolling to view additional content.

CT-03 – iPad Responsiveness

The application was tested using an iPad Pro 13-inch (M5) simulator. The interface maintained a constrained content width rather than expanding across the entire display. This preserved the readability of task descriptions and the accessibility of interactive controls.

Safari Web Inspector was used to examine the application's stored JSON data. The recorded task descriptions and completion states were consistent with the application's displayed information.

CT-04 – Simulated Storage Restriction

A storage-write failure was simulated through Safari Web Inspector by overriding the browser's Storage.prototype.setItem() method to throw a SecurityError.

When a new task was created, the application displayed a warning explaining that local storage was unavailable and that changes would not persist after refreshing the page. The task was nevertheless added to the interface, demonstrating that the application could continue operating using temporary in-memory storage.

This test demonstrated successful error handling under a simulated storage-write restriction. It did not establish behavior under complete unavailability of the localStorage API.

CT-05 – JavaScript Disabled

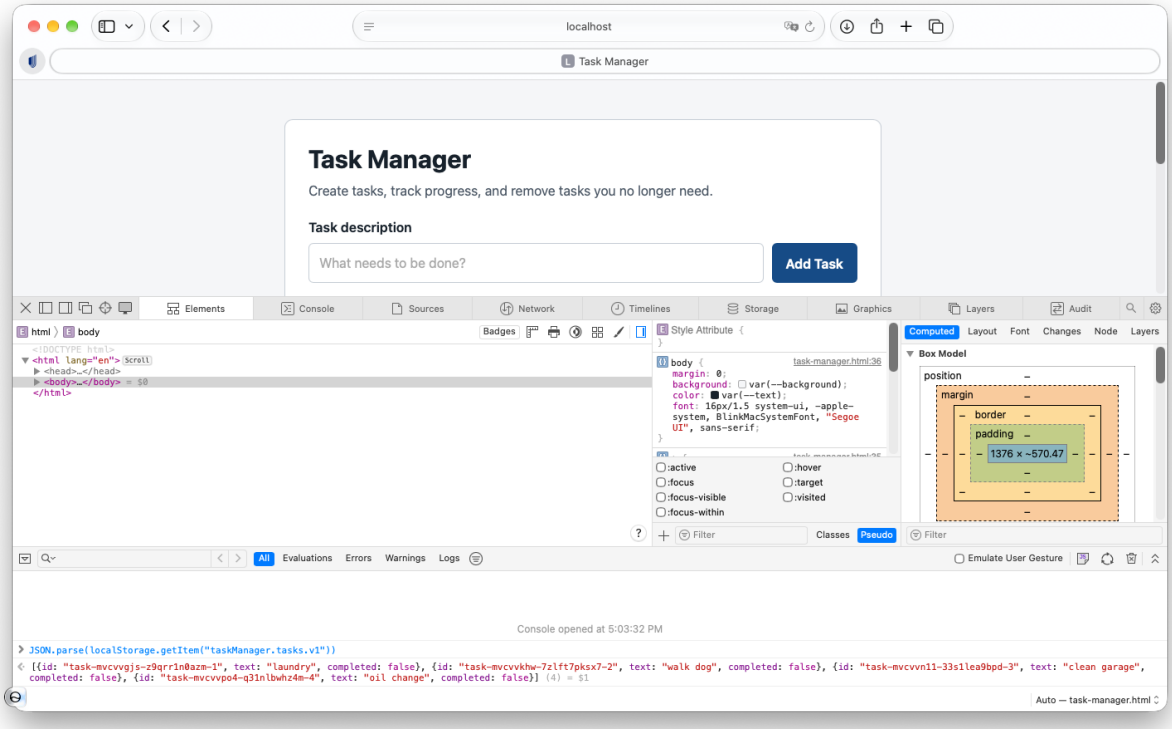
JavaScript execution was disabled in Safari to evaluate the application's behavior when its primary scripting functionality was unavailable.

The application displayed a warning informing users that JavaScript was required. Previously stored tasks were not rendered while JavaScript was disabled, but they appeared again after JavaScript was re-enabled.

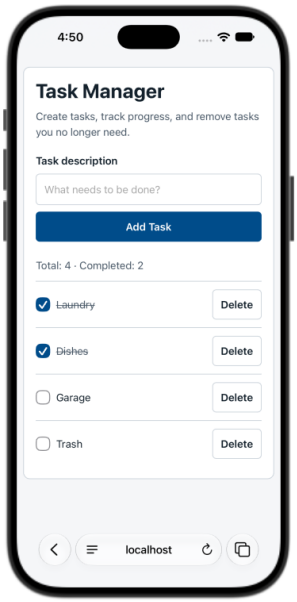
The task input field and Add Task button remained visible while JavaScript was disabled, despite being unable to perform their intended functions. This was identified as a potential low-severity usability issue.

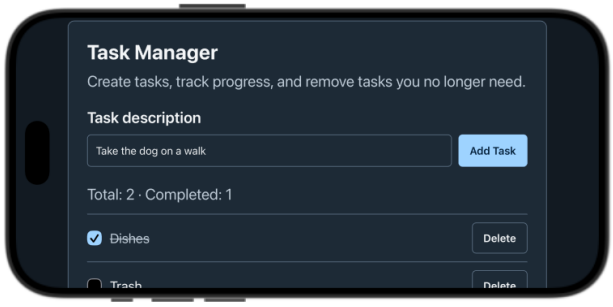
Documented Evidence

CT-01 – Documentation

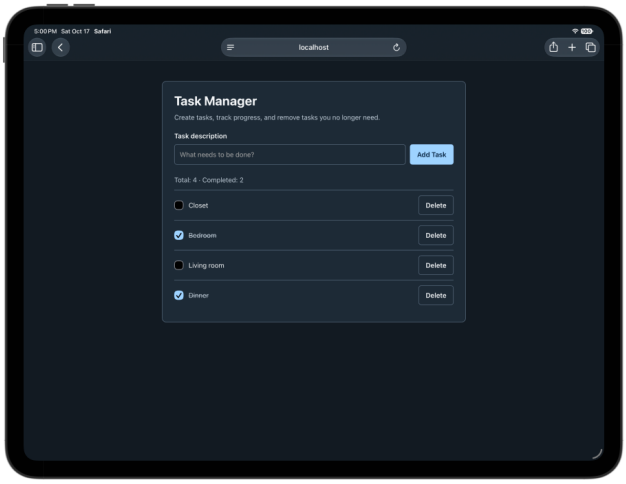


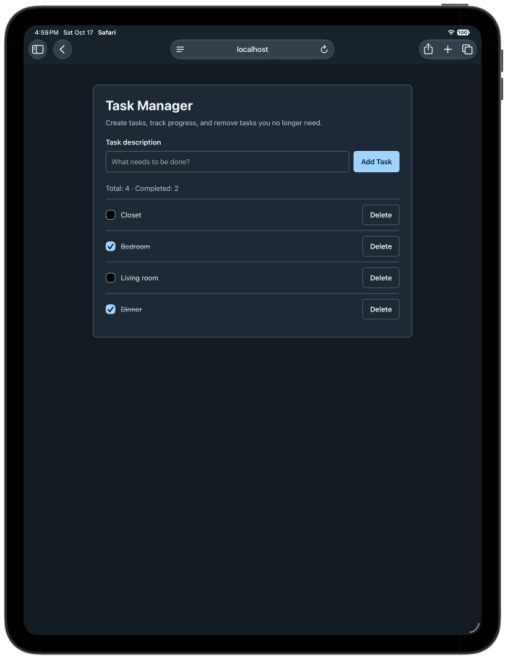
CT-02 – Documentation



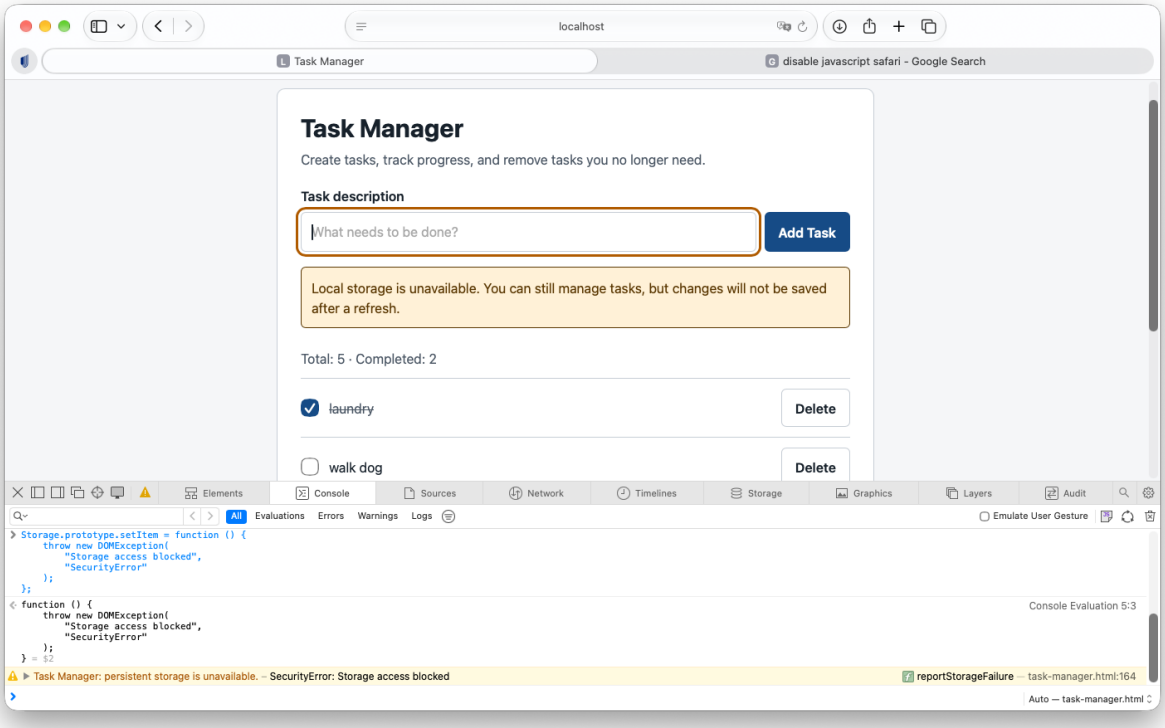


CT-03 – Documentation

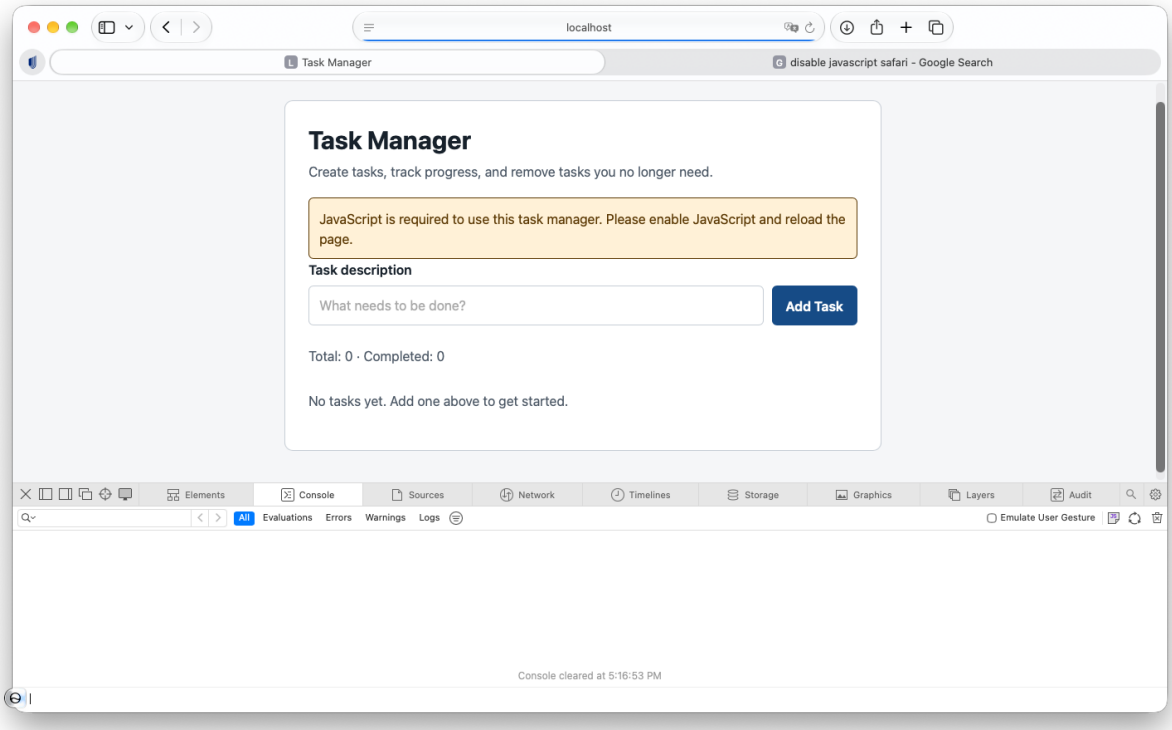




CT-04 – Documentation



CT-05 – Documentation



Issue Documentation

01 – Nonfunctional Control Remains Visible When JavaScript is Disabled

Field	Description
Configuration	CT-05 - Safari/macOS/JavaScript Disabled
Severity	Low
Classification	Usability Issue
Description	The application continues displaying its task-entry controls when JavaScript execution is disabled, despite those controls being unable to perform their intended functions.
Steps to Reproduce	Open the application in Safari, disable JavaScript through the Develop menu, and refresh the page.
Expected	The application should communicate that JavaScript is required and ideally disable or hide nonfunctional controls.
Actual	The application displays the required JavaScript warning, but the task input field and Add Task button remain visible and appear usable.
Impact	Users may attempt to interact with controls that cannot complete their intended operations.
Evidence	CT-05 screenshot showing the application with JavaScript disabled.

Assessment

The application satisfies its requirement to notify users when JavaScript is unavailable. However, leaving nonfunctional controls visible introduces a minor usability concern.

02 – Reduced Visible Content in Mobile Landscape Orientation

Field	Description
Configuration	CT-02 - Safari/ iOS 27.0 / iPhone 18 Pro / Landscape
Severity	Low
Classification	Usability Issue
Description	Rotating the simulated iPhone into landscape orientation substantially reduces the vertical space available for displaying tasks.
Steps to Reproduce	Open the application in the iPhone Simulator, create several tasks, and rotate the device into landscape orientation.
Expected	The application should maintain a usable layout and allow users to access all task controls.
Actual	The interface adjusts to the wider viewport, but fewer tasks are visible simultaneously, requiring additional vertical scrolling.
Impact	Users must scroll more frequently when managing multiple tasks in landscape orientation.
Evidence	CT-02 screenshots showing portrait and landscape layouts.

Assessment

This behavior does not violate the application's requirements. The layout remains responsive, and scrolling is normal browser behavior. The finding is documented as a usability limitation rather than a confirmed software defect.

03 – Task Persistence Unavailable When Storage Writes Are Blocked

Field	Description
Configuration	CT-04 — Safari / macOS / Simulated blocked localStorage writes
Severity	Low
Classification	Configuration limitation
Description	The application cannot persist newly created tasks when browser storage operations are blocked.
Steps to Reproduce	Open Safari Web Inspector and override `Storage.prototype.setItem()` to throw a `SecurityError`. Attempt to create a new task.
Expected	The application should handle the storage failure without crashing, notify the user, and continue operating with temporary data.
Actual	The application displayed a storage warning, logged the `SecurityError`, and continued accepting tasks in memory.
Impact	Changes made while storage writes are blocked cannot be preserved through localStorage.
Evidence	CT-04 screenshot showing the storage warning and Console output.

Assessment

This finding represents successful graceful degradation rather than a software defect. The application correctly handled the simulated storage restriction and informed the user of the persistence limitation.

Summary of Findings

The testing did not identify any high or medium severity defects. The most significant usability concern was the continued presentation of task-entry controls when JavaScript was disabled. Other findings reflected limitations of the selected configurations rather than failures of the application itself.

The storage restriction test was particularly useful because it demonstrated that the application could continue operating after a browser API failure. Rather than terminating unexpectedly, the application provided a warning and maintained temporary task data.

Reflection and Conclusion

1. What was the most significant configuration issue?

localStorage restrictions where the most significant. When storage writes were blocked, the application could no longer persist new tasks. However, it handled the error appropriately by displaying a warning and continuing to operate using temporary memory.

2. Which testing tool was most useful?

Safari Web Inspector was the most useful tool because it allowed direct inspection of stored JSON data, JavaScript execution, and error handling. Xcode Simulator complemented this by allowing testing across different devices and screen orientations.

3. Which three browser configurations would your prioritize?

I would prioritize Chrome on Windows, Safari on iOS, and Firefox on Windows or Linux. These configurations provide coverage across three major browser engines and both desktop and mobile environments.

4. How would you prioritize fixes?

I would first address the nonfunctional controls displayed when JavaScript is disabled. Next, I would review mobile landscape usability. Finally, I would consider improving the warning displayed when localStorage becomes unavailable. None of these findings represents a critical defect.

5. What additional testing would you perform?

I would expand testing to Chrome, Firefox, and Edge, investigate older browser versions, and evaluate additional viewport sizes. I would also test complete localStorage unavailability and offline operation.

Conclusion

The application successfully performed its primary functions across the tested Safari configurations. Testing demonstrated responsive behavior across devices and appropriate handling of restricted browser capabilities. Although no critical defects were identified, the exercise highlighted the importance of testing environmental dependencies and identifying opportunities for improved usability.